o/o CUTLINE

CUTLINE, explained.

From “I know nothing about this project” to understanding every part of it: what it is, why it matters, how it works, what’s already built, and what’s left before the deadline.

> **THE ONE-LINE PITCH**
>
> CUTLINE removes dangerous permissions without breaking the work that legitimate users and bots still need to do, and proves the repair actually worked.

Snapshot taken
16 Oct 2026, 17:35 IST
(project is being actively worked on)

Hackathon
FalkorDB × WeMakeDevs Graph Hacks
Track 01 · Agents That Act on Connected Data

Submission deadline
18 Oct 2026, 23:59 IST
about 2 days and 6 hours from this snapshot

CONTENTS

How this document is organised

Each part goes one level deeper than the one before. Part 1 assumes nothing. By Part 5 you are reading the actual database queries. You can stop at any part and still have a complete picture at that level of detail.

1 **The idea in plain English** — Level 1
The keycard analogy, the problem, and why CUTLINE is different

2 **The hackathon** — Level 1
The event, the rules that matter, and how CUTLINE fits Track 01

3 **The vocabulary** — Level 2
Every technical word in this project, explained from scratch

4 **The demo story, step by step** — Level 2
Maya, Felix, the DSAR bot, and the repair that keeps everyone working (with screenshots)

5 **How it works under the hood** — Level 3
Architecture, graph model, Cypher, planner, verification, staleness, agent, security

6 **A tour of the codebase** — Level 3
Which folder and file does what, and the tech stack

7 **What is built right now** — Status
Status board, test results, benchmark results, live end-to-end run

8 **What is left, and the plan to the deadline** — Plan
Must-do items, known issues, stretch goals, day-by-day plan, risks

9 **Running it yourself** — Practical
Exact commands for this Windows machine

10 **Presenting it to judges** — Practical
The 3-minute demo script and answers to the questions judges will ask

Have 10 minutes?
Read Part 1 and the status board in Part 7.

Need to demo it?
Read Parts 1, 4, 9 and 10.

Need to defend it?
Read everything; judges may ask about Part 5.

PART 1

The idea in plain English LEVEL 1

CUTLINE in one sentence

You tell CUTLINE "this person must *not* be able to do X, but these people must *still* be able to do Y and Z." It finds every hidden way the person can currently do X, works out the smallest change to the permission setup that removes all of them without breaking Y and Z for anyone, tests that change against a running app, and only applies it after a human signs off.

An analogy: the office keycard problem

Imagine an office building where every door is opened by a keycard. Nobody programs cards door by door; that would be unmanageable. Instead there are **access groups**. The "Support" group opens the order desk and the reports room. Groups can also **include other groups**: when someone set up Support, they made it include a general-purpose "Shared Ops" group, and later an "Export Helper" group.

Here is the trouble. Both Shared Ops and Export Helper open the **customer records vault**. So Maya, a support agent, can walk into the vault, even though nobody ever deliberately gave her that access. It came along for the ride, two separate ways.

WHY YOU CAN'T JUST FIX IT THE OBVIOUS WAY

- **Take the vault off Shared Ops?** Felix in Finance uses Shared Ops to get into the vault for month-end close. Finance breaks. And Maya still gets in through Export Helper.
- **Remove Shared Ops from the Support group?** Maya still gets in through Export Helper.
- **Remove both from Support?** The DSAR bot, an automated courier that legally has to fetch customer records when a customer asks for their data, is also in Support and needs that vault. A legally required process breaks.

The right fix is structural: create a new, narrow group, "Support (safe)", that opens exactly the order desk and the reports room and nothing else. Move Maya and her colleague Noah onto it, and take them out of the old Support group. The DSAR bot stays where it is, Felix is untouched, and the vault is closed to support agents through *both* routes.

A careful facilities manager would then do three more things, and CUTLINE does all three:

1. **Actually try the doors.** Don't trust the paperwork. Swipe Maya's card at the vault (should fail), at the order desk (should work), and swipe Felix's and the bot's cards at the vault (should work).
2. **Get a sign-off before changing anything.** Nothing is changed until a human approves.
3. **Keep a dated logbook.** The sign-off is tied to the exact state of the building plan. If someone changes the groups afterwards, the old sign-off is no longer valid. CUTLINE calls this **stale** and refuses to apply a stale plan.

In software, the keycards are **permissions**, the groups are **roles**, "group includes group" is **role inheritance**, and the doors are **API endpoints** in an application.

Why this is a real problem

Real companies have thousands of users, service accounts, and now AI agents, attached to hundreds of roles that inherit from each other. Over time, access piles up ("permission sprawl"). Everyone knows the principle of **least privilege**: give each identity only what it needs. Excess access persists anyway, because removing it is scary. Nobody can see every hidden path, and nobody wants to be the person who broke Finance's month-end close. CUTLINE takes away that fear: it shows every path, predicts every consequence, and proves the outcome.

What makes CUTLINE different

Tools that *find* excessive access already exist. AWS IAM Access Analyzer computes effective permissions and complex access paths. AuthZed's SpiceDB Playground Assistant can explain why a permission is granted. So "AI + graphs + permissions" on its own is not new. CUTLINE asks a different question:

> Existing tools answer "can X do Y?" CUTLINE answers "how do I change the structure so X can no longer do Y, without breaking anything X or anyone else still has to do?", and then proves it against a running application.

The defensible combination is: **constraint-driven structural repair + business-workflow preservation + rejecting bad fixes with concrete counterexamples + independent runtime verification + evidence that expires when the policy changes.**

The six things CUTLINE does

1	2	3	4	5	6
Discover	**Contract**	**Plan**	**Verify**	**Approve & apply**	**Stay honest**
Find every route from a person to a forbidden capability.	Write down what must be denied and what must keep working.	Generate candidate fixes; test each on a throw-away copy; reject bad ones with proof.	Send real requests to a running app and check the answers.	A human approves; the change lands all-at-once or not at all.	If the policy changes, old proof expires; CUTLINE re-plans.

PART 2

The hackathon LEVEL 1

The event

Name	Graph Hacks: Context for AI Agents, by FalkorDB × WeMakeDevs
Window	15 Oct 2026, 00:01 IST → **18 Oct 2026, 23:59 IST**
Team size	Up to 4 people
Track entered	**Track 01: Agents That Act on Connected Data**. Agents that do multi-hop reasoning, use graph queries/algorithms as tools, select tools through MCP, act on what the graph shows, and explain their results.
Must submit	Public code repository · the graph model · the Cypher queries / graph algorithms the product relies on · a working demo video · a live deployment *or* complete local setup instructions

The rules that matter most

1. FalkorDB must do central work

FalkorDB has to be the primary graph database. If the product still works with FalkorDB removed, the graph counts as decoration and the project does not qualify. CUTLINE has a test that proves it fails without FalkorDB (see Part 5).

2. Built during the window

Ideas, notes and diagrams may predate the event; the main coding must happen 15–18 Oct. The CUTLINE workspace was empty at the start of the build (16 Oct).

3. Disclose AI help, and understand it

AI coding tools are allowed, but you must disclose them and be able to explain the architecture, queries, and decisions yourself. **This document exists largely for that reason.**

4. Synthetic or own data only

No private or personal data. CUTLINE uses a made-up "Customer Operations" company with 1,000 synthetic customer records.

A CAUTION ON COMPETITION SIZE

The planning report could not verify any entrant count for this specific hackathon. WeMakeDevs reported 20,000+ participants across four *combined* February 2026 hackathons. Don't put "20,000 competitors" in the pitch. Do assume the field is crowded.

How CUTLINE maps onto Track 01

TRACK 01 ASKS FOR…	…AND CUTLINE SHOWS IT BY
Multi-hop reasoning	Tracing `person → role → inherited role(s) → capability` routes up to 6 inheritance hops deep, including several independent routes to the same capability
Graph queries as agent tools	11 typed tools (trace access, find risks, propose repair, …) that run fixed, parameterised Cypher on FalkorDB

TRACK 01 ASKS FOR…	…AND CUTLINE SHOWS IT BY
Tool selection through MCP	The same 11 tools are exposed as an MCP server; an AI model picks which to call
Actions based on the graph	Repairs are generated from the discovered paths and applied to the live policy graph after approval
Explainable results	Every rejected fix carries a witness path or the named workflow it would break; every approval carries 12 real HTTP results

The planning report recommends entering **only** Track 01. Tracks 02 (agent memory) and 03 (company knowledge) aren't CUTLINE's core, and bolting on features to fit them would dilute the story.

Where the plan came from

A deep evaluation report ("CUTLINE: Deep Evaluation…") scored the idea **84/100 as specified**, with a realistic **91–93/100 ceiling** if it is built narrow and deep. Its main warning was **scope explosion**: "Do not build an authorization platform. Build one spectacular permission-repair transaction." The build followed that advice: one scenario, one policy model, depth over breadth. The report's "never cut" list was: path tracing in FalkorDB, structural repair, counterexample rejection, real runtime verification, and human-reviewed change. All five are built (Part 7).

PART 3

The vocabulary LEVEL 2

Every technical term used in the rest of this document, explained without assuming background. Terms are grouped so each builds on the ones before it.

Graphs and FalkorDB

Graph

Data stored as **things** (nodes) and **connections** between them (relationships). A map of who-is-connected-to-what, rather than rows in a spreadsheet.

Node

One thing in the graph: a person (Maya), a role (Support), a capability (export_customers). Nodes have a **label** (its type) and **properties** (e.g. `department: support`).

Relationship (edge)

A directed, named connection: `Maya –ASSIGNED_TO→ Support`. Direction matters.

Path / multi-hop

A chain of relationships. "Maya → Support → Shared Ops → export_customers" is a 3-hop path. Answering questions by following chains is **multi-hop reasoning**.

FalkorDB

The sponsor's graph database (runs as a Redis module). CUTLINE uses a hosted instance, **FalkorDB Cloud** (AWS Mumbai, ap-south-1).

Cypher

The query language for graphs. `MATCH (a)-[:KNOWS]->(b)` means "find a and b where a KNOWS b". Like SQL, but for paths.

Variable-length match

`[:INHERITS*0..6]` means "follow 0 to 6 INHERITS relationships". This is how one query finds routes of any length up to a limit.

GRAPH.COPY

A FalkorDB command that clones a whole graph. CUTLINE clones the policy once per candidate fix, so each fix is tested on its own throw-away copy.

Permissions

Principal

Anyone or anything that acts: a human (Maya) or a service identity (DSAR Bot). Called `User` in the graph.

Role

A named bundle of permissions you assign to principals instead of granting one by one.

Capability

One thing you can do in the app, e.g. `export_customers`, served by `POST /demo/customers/export`.

RBAC

Role-Based Access Control: principals get roles, roles grant capabilities. CUTLINE supports allow-only RBAC with role inheritance.

Role inheritance

`Support –INHERITS→ Shared Ops` means Support receives everything Shared Ops grants. Inheritance can chain, which is how access hides.

Effective access

What a principal can *actually* do once every inherited role is followed, not just what was directly granted.

Least privilege

The security principle that every identity should have only the access it needs.

DSAR

Data Subject Access Request: a customer's legal right (e.g. under GDPR) to get a copy of their data. The DSAR Bot fulfils these, so it legitimately needs export.

CUTLINE's own concepts

Guardrail (must_deny)
A security rule that must hold, stored as a graph node: "support agents must not bulk-export customers".

Workflow (must_allow)
A business process that must keep working, also stored as a node: Order support, Finance close, DSAR fulfilment.

Contract
The full list of (principal, capability) pairs that must be denied and that must stay allowed. Derived from guardrails and workflows, optionally extended by an operator in plain English.

Candidate (repair)
One possible fix: a list of edits such as "unassign Maya from Support".

Counterexample / witness
Concrete proof that a candidate is wrong: the exact path that is still open, or the exact workflow that breaks.

Runtime verification
Checking a fix by sending real HTTP requests to a running app and reading the actual 200 (allowed) / 403 (denied) answers.

Enforcement point
The code inside the app that decides allow/deny on each request. In CUTLINE it is deliberately separate from the planner, so it can check the planner's work.

OPA / Rego
Open Policy Agent, an industry-standard policy engine; Rego is its policy language. CUTLINE can use OPA as its independent enforcement engine.

Revision & hash
The policy has a version number (r1, r2, …) and a fingerprint (`sha256:…`) of its contents. Evidence is tied to both.

Stale
Evidence whose revision or fingerprint no longer matches the live policy. A stale plan cannot be approved or applied.

Software and AI

Atomic write
A change that happens entirely or not at all. CUTLINE applies a whole repair in one Cypher statement, so the policy is never left half-changed.

Compare-and-set
"Only write if the revision is still the one I planned against." This prevents applying a fix to a policy that changed in the meantime.

Regression test
An automated check that keeps a fix true later, e.g. `test_maya_cannot_export_customers`. CUTLINE exports these as runnable pytest code.

API / endpoint
A URL a program calls, e.g. `GET /api/risks`. FastAPI is the Python framework serving CUTLINE's API.

LLM agent
A large language model (here, Claude) that works in a loop: decide which tool to call, read the result, decide again.

Tool
A function the agent may call, with a fixed input shape. CUTLINE's tools never let the model write raw database queries.

MCP
Model Context Protocol, a standard way to expose tools to any AI client (Claude Code, Claude Desktop, …).

Deterministic
Same input, same output, every time. CUTLINE's planner and verifier are deterministic; the AI never decides whether a fix is safe.

PART 4

The demo story, step by step LEVEL 2

The whole product is demonstrated on one carefully designed scenario, a fictional "Customer Operations" team. It is small enough to follow by eye and still contains every trap that makes permission repair hard.

The cast

WHO / WHAT	TYPE	THEIR SITUATION
Maya Chen, **Noah Patel**	Humans · Support	Need to view orders and generate reports. Must **never** bulk-export customer data. Currently **can**, through two routes.
Felix Romero	Human · Finance	Legitimately needs `export_customers` for the **Finance close** workflow. Gets it via Shared Ops.
DSAR Bot	Service identity	Legitimately needs `export_customers` for **DSAR fulfilment**. It is in the Support role, which is why "just strip export from Support" fails.
Support	Role	Grants view_orders and generate_reports directly; inherits Shared Ops and Export Helper.
Shared Ops	Role	A shared bundle used by both Support and Finance. Grants export_customers.
Export Helper	Role	Attached to Support so the DSAR bot could export. Grants export_customers.
Finance	Role	Assigned to Felix; inherits Shared Ops.

Design note: the DSAR Bot was added on purpose. Without it, "detach both bundles from Support" would be a valid, cheaper fix and the clever structural repair would only win by rigging cost weights. With it, the scoped-role migration is honestly the only valid repair.

The graph, before the repair

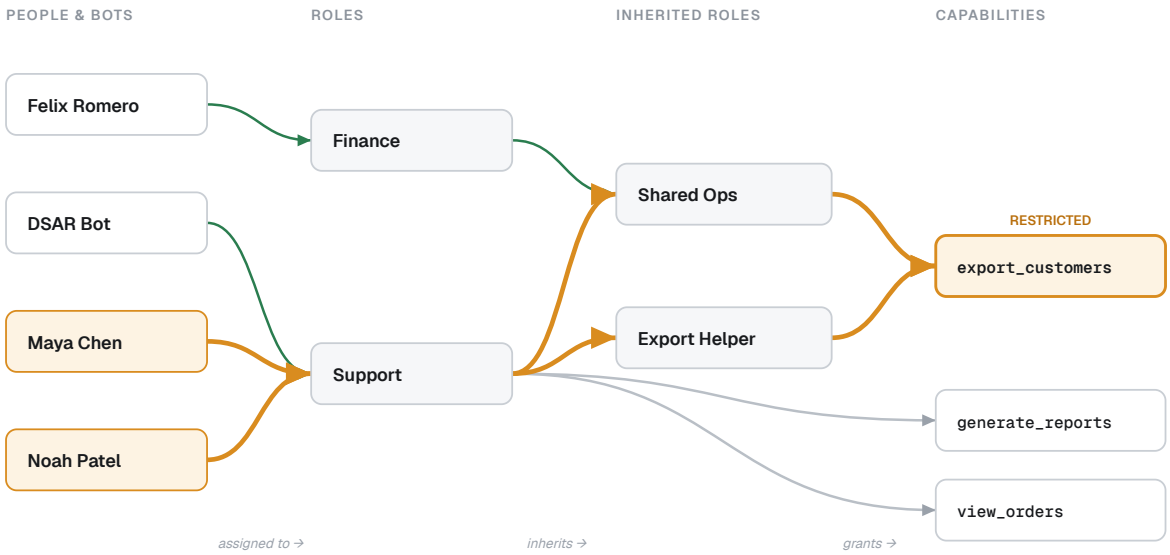


Figure 1. The policy graph at the start. **Amber** = the routes by which Maya and Noah reach `export_customers`: through Shared Ops *and* through Export Helper (2 routes × 2 people = 4 risky paths). **Green** = the legitimate routes Felix and the DSAR Bot depend on. They share edges with the amber routes, and that overlap is the whole difficulty.

The breach, in one request

```
POST /demo/customers/export      X-Cutline-Principal: maya      →  200 OK · 1,000 customer records
```

The contract: what must be true afterwards

CUTLINE reads the business intent straight out of the graph: one guardrail and three workflows. Together they produce **2 must-deny pairs and 6 must-allow pairs:**

MUST BE DENIED (GUARDRAIL NO_SUPPORT_BULK_EXPORT)		MUST STAY ALLOWED	BECAUSE OF WORKFLOW
Maya	`export_customers`	Maya, Noah × `view_orders`, `generate_reports`	Order support
Noah	`export_customers`	Felix × `export_customers`	Finance close
		DSAR Bot × `export_customers`	DSAR fulfilment

An operator can also type a requirement in plain English ("Support agents must not export customer data, but keep order lookup and reporting"). CUTLINE parses it into the same typed pairs, flags phrases it couldn't understand, and asks for confirmation before using it. Workflows declared in the graph are always kept, so a sloppy sentence can't silently drop Finance.

The obvious fixes, and why each one fails

CUTLINE generates candidates (10 of them in this scenario) and tests every single one on its own copy of the graph in FalkorDB. Here are the instructive ones:

CANDIDATE FIX	RESULT	COUNTEREXAMPLE FALKORDB FOUND ON THE COPY
Revoke `export_customers` from Shared Ops	REJECTED	Felix loses export → **Finance close breaks**. And Maya still exports via Export Helper.
Detach Shared Ops from Support	REJECTED	Still open: `maya → support → export_helper → export_customers`
Revoke export from Export Helper / detach Export Helper	REJECTED	Still open: `maya → support → shared_ops → export_customers`
Detach *both* bundles from Support	REJECTED	DSAR Bot loses export → **legally required DSAR fulfilment breaks**
Remove Maya and Noah from Support	REJECTED	They lose `view_orders` and `generate_reports` → **Order support breaks**
Create "Support (safe)" with exactly view_orders + generate_reports; move Maya and Noah onto it	PASS	All 8 constraints hold. No new access for anyone. Chosen.

The winning repair: 7 edits, cost 8

```
1  CREATE_ROLE     support_safe
2  ADD_GRANT       support_safe → generate_reports
3  ADD_GRANT       support_safe → view_orders
4  ASSIGN_ROLE     maya → support_safe
5  ASSIGN_ROLE     noah → support_safe
6  UNASSIGN_ROLE   maya ✕ support
7  UNASSIGN_ROLE   noah ✕ support
```

Cost = 2 (creating a role) + 6 × 1 (each other edit) = **8**. Cheaper candidates exist, but every one of them fails a constraint, so CUTLINE picks the **cheapest candidate that passes**.

One residual note CUTLINE reports honestly: the old Support role still reaches export, now held only by the DSAR Bot. Anyone newly assigned to Support in the future would inherit export, so review it before reuse.

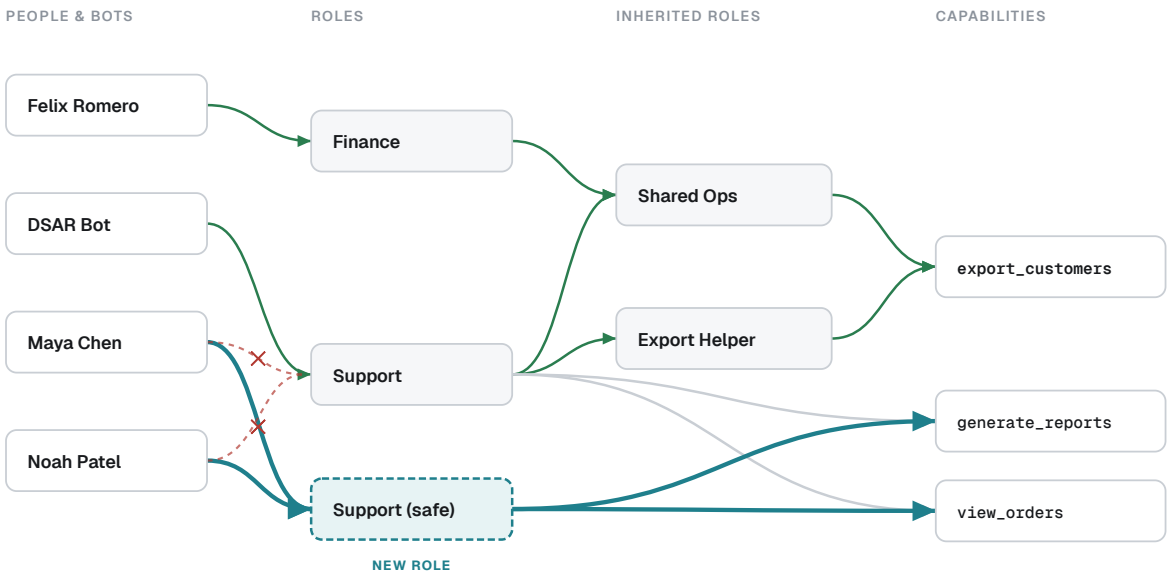


Figure 2. After the repair. **Teal** = new: the Support (safe) role and its grants and assignments. **Red dashed** ✕ = removed assignments. Maya and Noah no longer have any route to `export_customers`; Felix and the DSAR Bot keep theirs untouched.

The proof: 12 real requests

Before anything is applied, CUTLINE sends real HTTP requests to a small demo app (the "Customer Ops API") twice: once against the **current** policy and once against the **candidate** policy. Every principal × capability combination is checked, 12 in total:

GROUP	REQUEST	BEFORE	AFTER (CANDIDATE)	RESULT
Must be denied (2)	Maya → `POST /demo/customers/export`	200 allowed	403 denied	✓
	Noah → `POST /demo/customers/export`	200 allowed	403 denied	✓
Must keep access (6)	Felix → export · DSAR Bot → export	200 · 200	200 · 200	✓✓
	Maya → `GET /demo/orders` · `GET /demo/reports`	200 · 200	200 · 200	✓✓
	Noah → orders · reports	200 · 200	200 · 200	✓✓

GROUP	REQUEST	BEFORE	AFTER (CANDIDATE)	RESULT
Unchanged (4) not in contract; must not change	Felix → orders · reports	403 · 403	403 · 403	✓✓
	DSAR Bot → orders · reports	200 · 200	200 · 200	✓✓

A check passes only if the app's real answer matches **both** what the contract demands **and** what FalkorDB predicted. The "unchanged" rows guard against a fix quietly granting or removing access nobody asked about.

Approve, apply, and break it again

1. **Approve & apply.** A human types their name and ticks "I reviewed these edits". The 7 edits land in one atomic database statement; the policy revision goes up by one (e.g. r3 → r4); the 12 checks re-run against the live policy and pass.
2. **Export the regression tests.** 12 named tests (e.g. `test_maya_cannot_export_customers`) as JSON or as a runnable pytest file.
3. **Live challenge.** Someone attaches a new "Contractor Tools" role (which grants export) to Maya's *new* role. The revision moves again (e.g. r4 → r5). Instantly: the old evidence is marked **stale**, "Apply" is blocked, FalkorDB shows the new forbidden path, and re-planning finds a 1-edit fix that cuts only the new connection. This is the moment the planning report called the "winning move": the system survives a change it did not anticipate.

PART 4, CONTINUED

What it looks like

Every screenshot below except Figure 5 was captured on 16 Oct 2026 at 17:31 IST by the automated browser walk (`e2e-live.mjs`) against the real API and FalkorDB Cloud. Figure 5 comes from the design-review run on mock data. The revision numbers in the screenshots (r13–r15) are higher than in the story above because the revision counter keeps counting up across scenario resets, and the scenario had been reset and replayed many times by then.

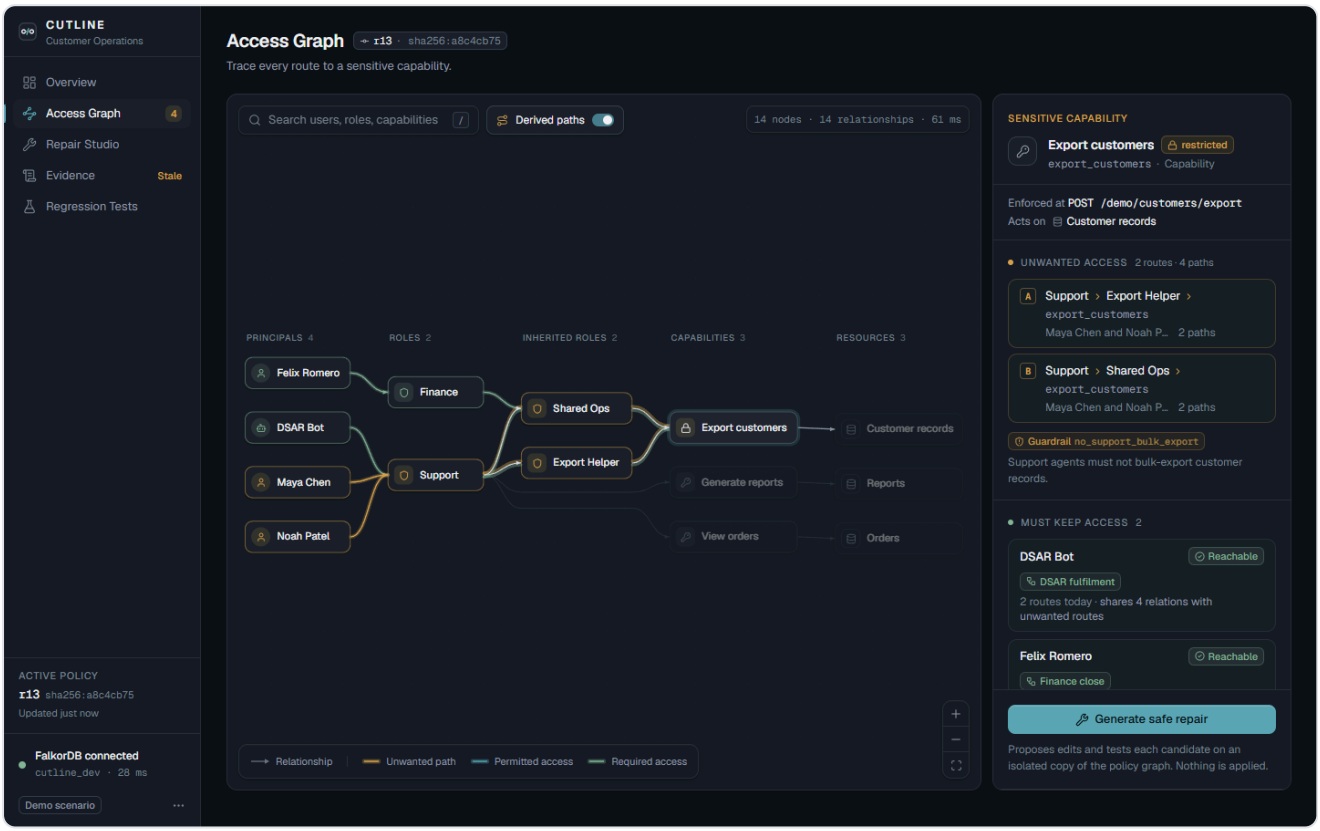


Figure 3. Access Graph. Selecting `export_customers` shows the two unwanted routes (A: via Export Helper, B: via Shared Ops), the guardrail that forbids them, and the "must keep access" principals (DSAR Bot, Felix) whose routes share relations with the unwanted ones.

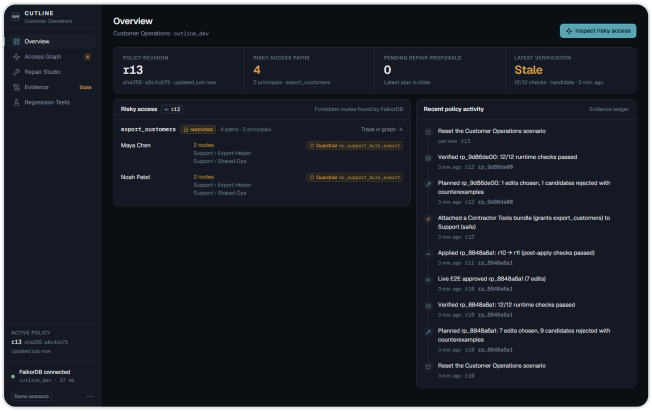


Figure 4. Overview (after several runs): policy revision, risky paths, pending proposals, latest verification, and the activity log, which is stored in FalkorDB as the evidence ledger.

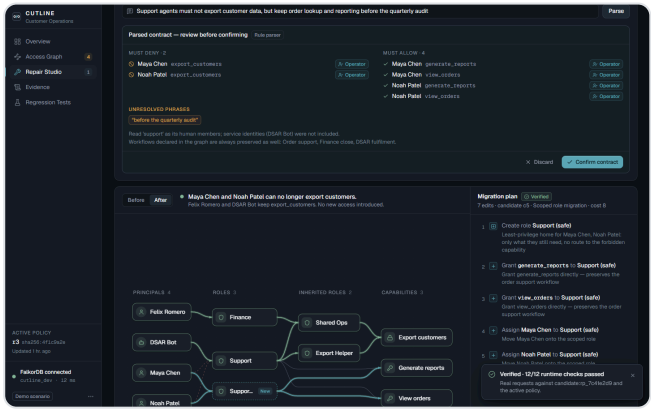


Figure 5. Plain-English contract (design-review run, mock data): parsed pairs, an unresolved phrase ("before the quarterly audit"), and an explicit Confirm step.

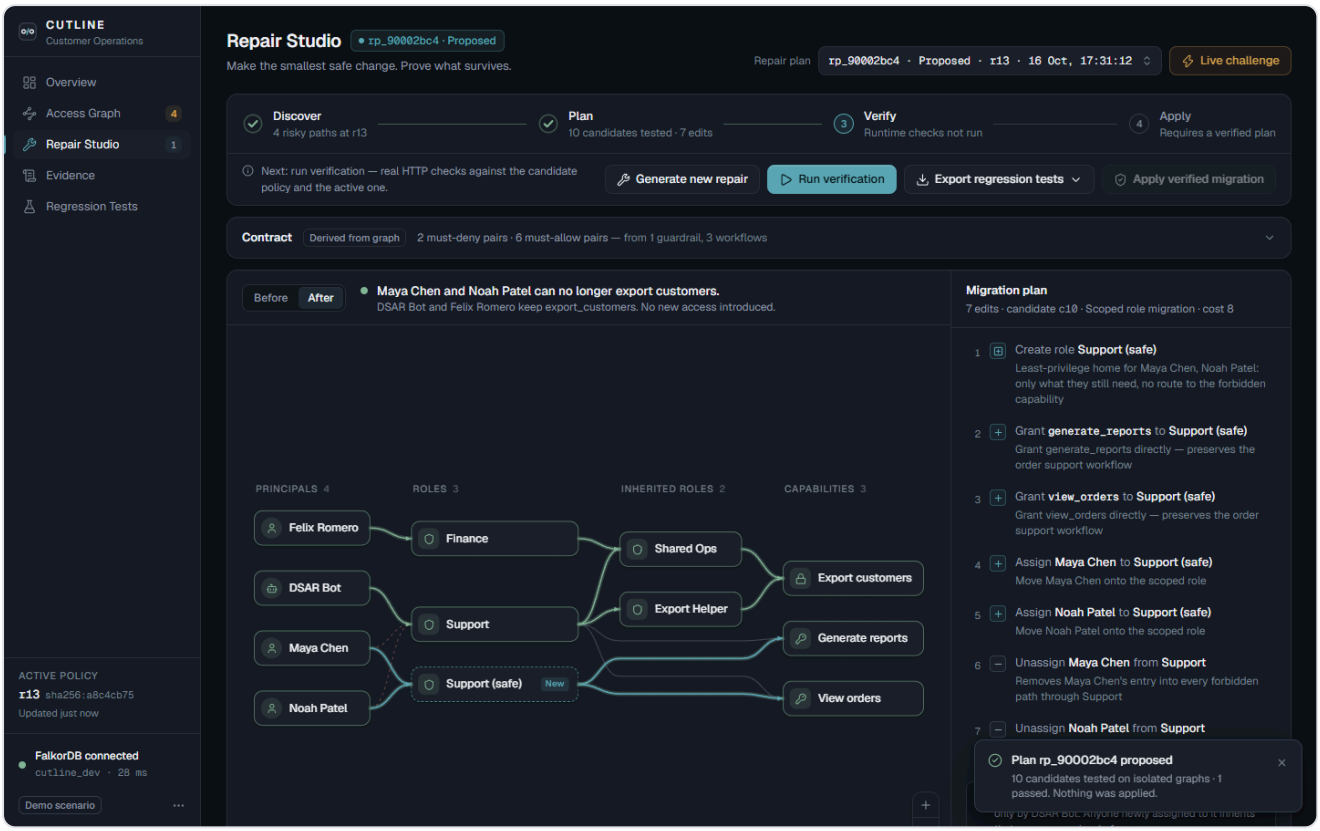


Figure 6. Repair Studio, plan proposed. The 4-stage progress bar (Discover → Plan → Verify → Apply), the "After" view with the new Support (safe) role, and the 7-edit migration plan with a reason for each edit. Nothing has been applied at this point.

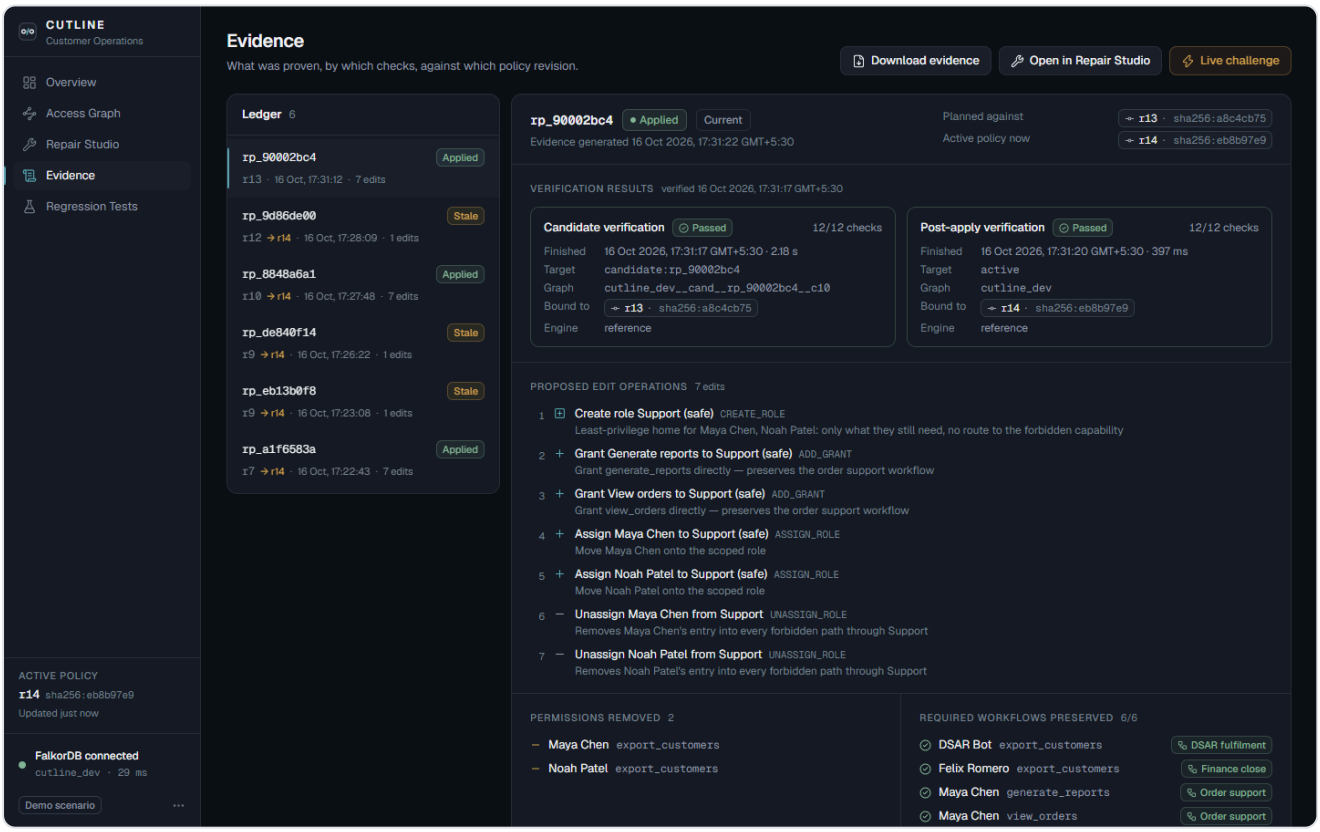


Figure 7. Evidence. Two verifications side by side. The candidate verification ran on a throw-away candidate graph (`cutline_dev__cand__…`) and is bound to the revision the plan was made on. The post-apply verification ran on the live graph `cutline_dev` and is bound to the new revision. Below them: the edits, the permissions removed, and the workflows preserved.

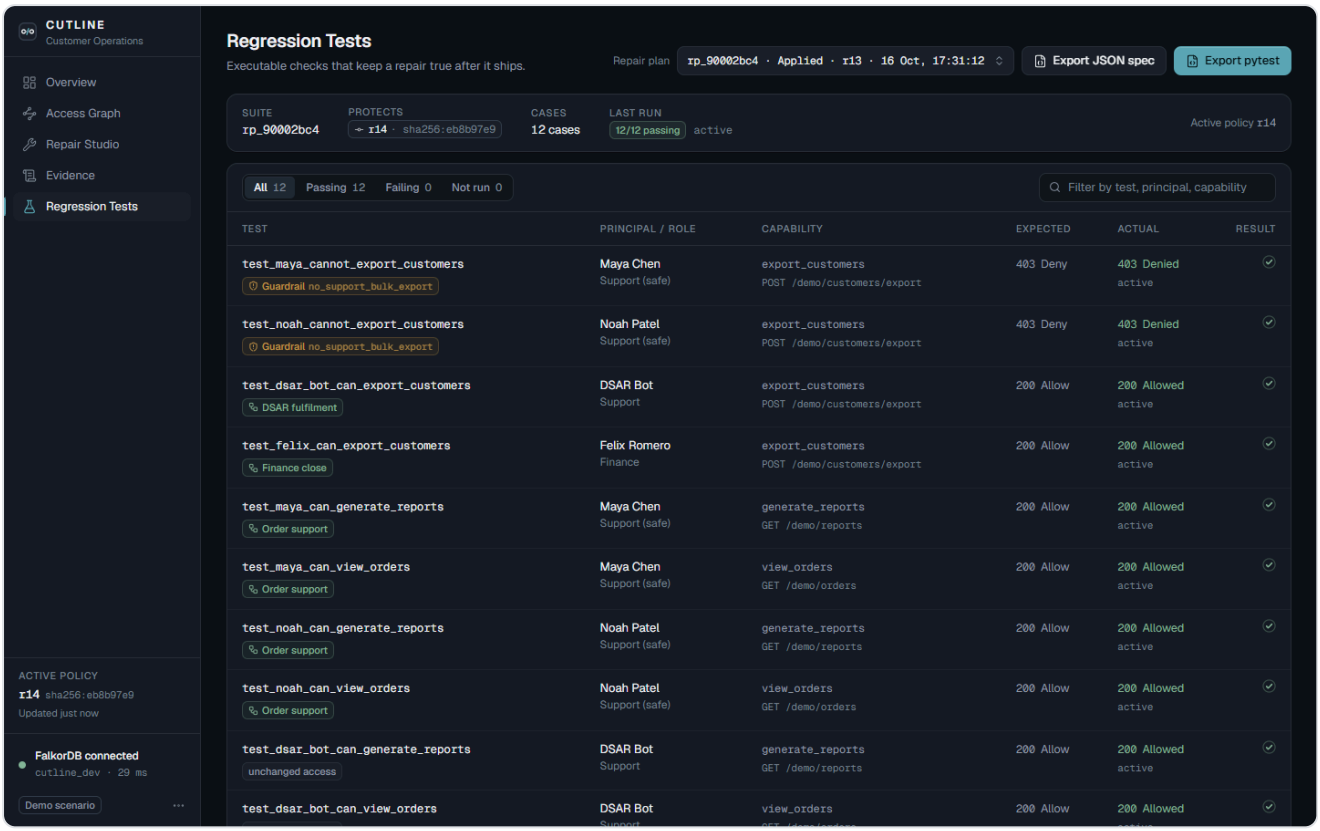


Figure 8. Regression Tests. 12 named cases, each tagged with where it came from (guardrail, workflow, or unchanged access), with expected and actual results. Exportable as a JSON spec or an executable pytest file.

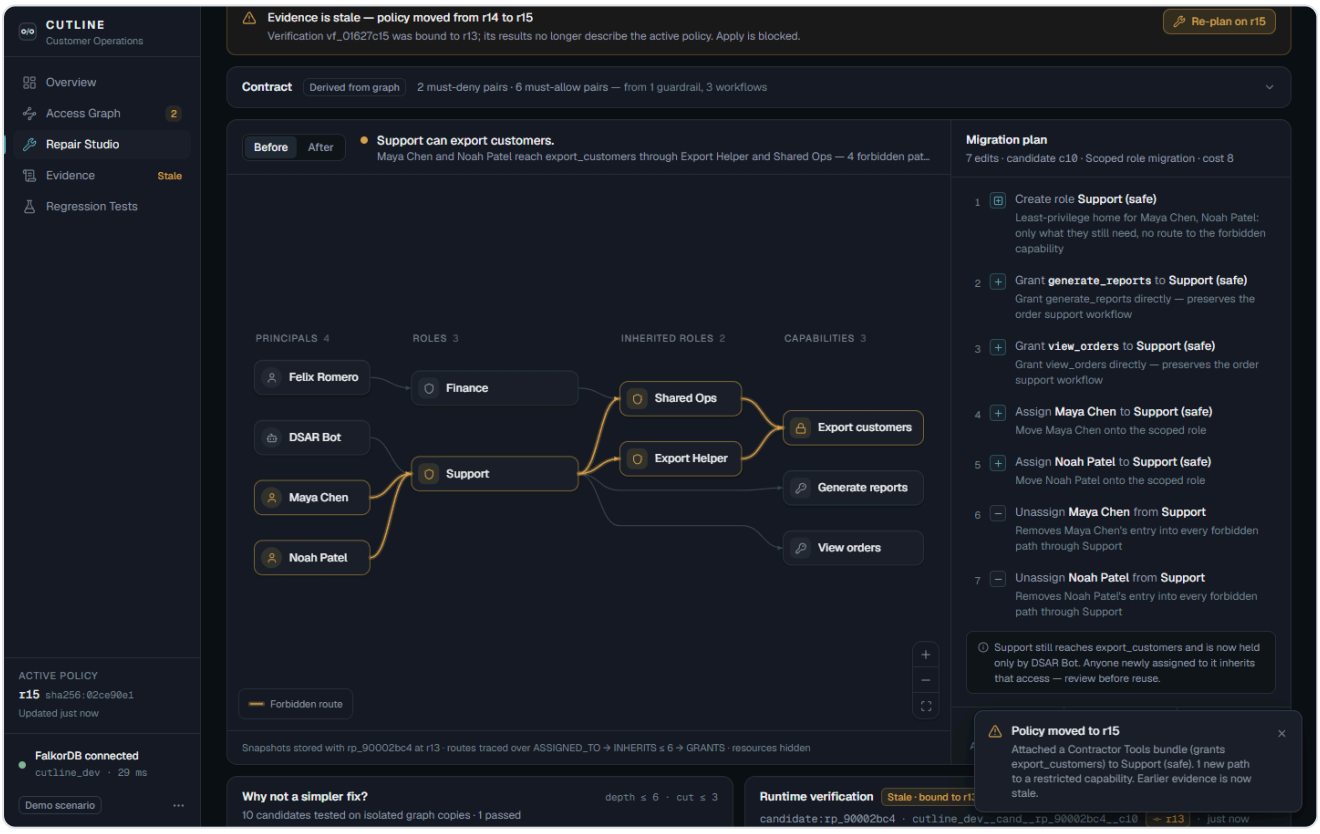


Figure 9. The live challenge. The banner reads “Evidence is stale: policy moved” to a new revision; the verification was bound to an older one, “its results no longer describe the active policy. Apply is blocked.” A “Re-plan” button is offered. Right: the residual-risk note about the old Support role. Bottom right: the toast describing the Contractor Tools change that caused it.

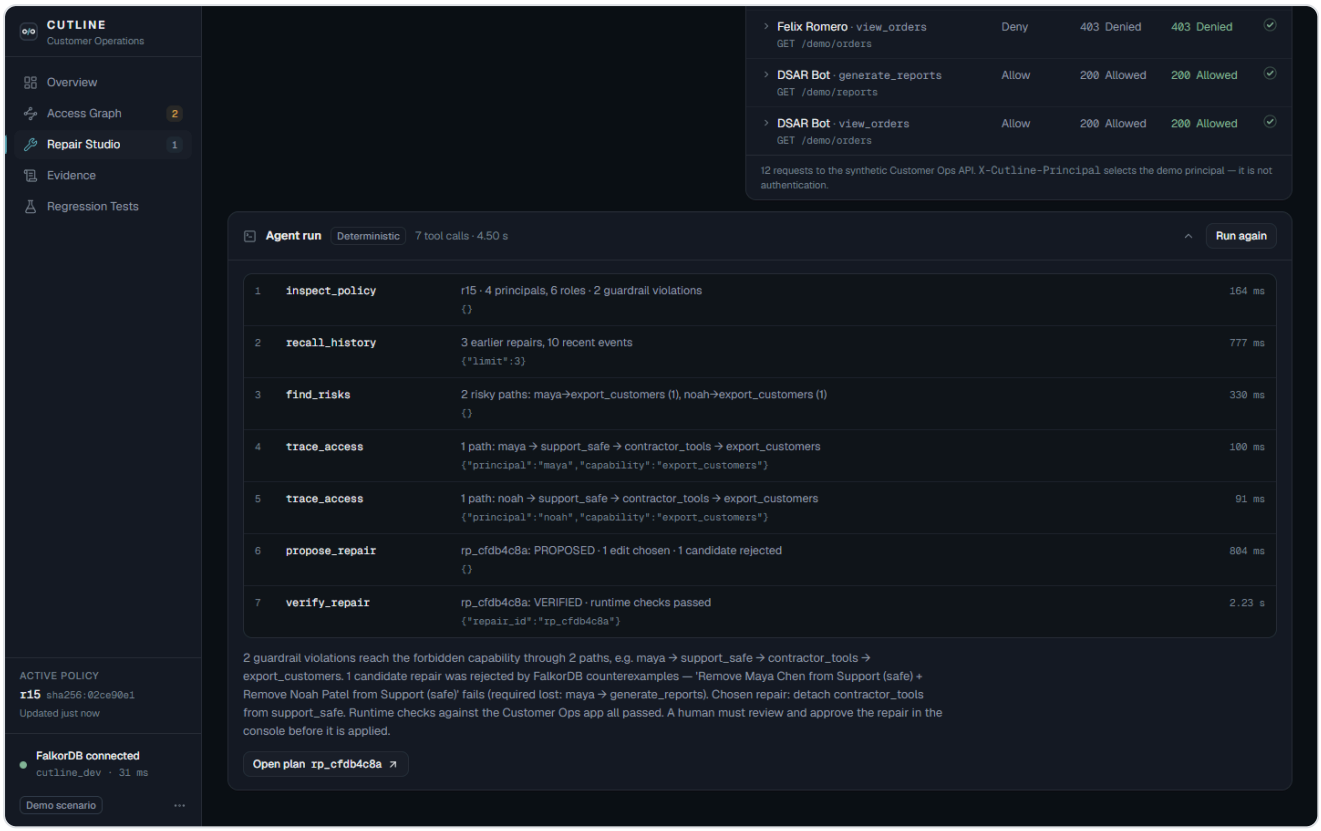


Figure 10. Agent run (deterministic mode, after the live challenge): 7 tool calls: `inspect_policy`, `recall_history` (reads earlier repairs from the FalkorDB ledger), `find_risks`, `trace_access` for Maya and for Noah (each finds the new path through Contractor Tools), `propose_repair`, `verify_repair`. The narrative explains which candidate was rejected and why, and ends: “A human must review and approve the repair in the console before it is applied.”

PART 5

How it works under the hood LEVEL 3

5.1 The big picture

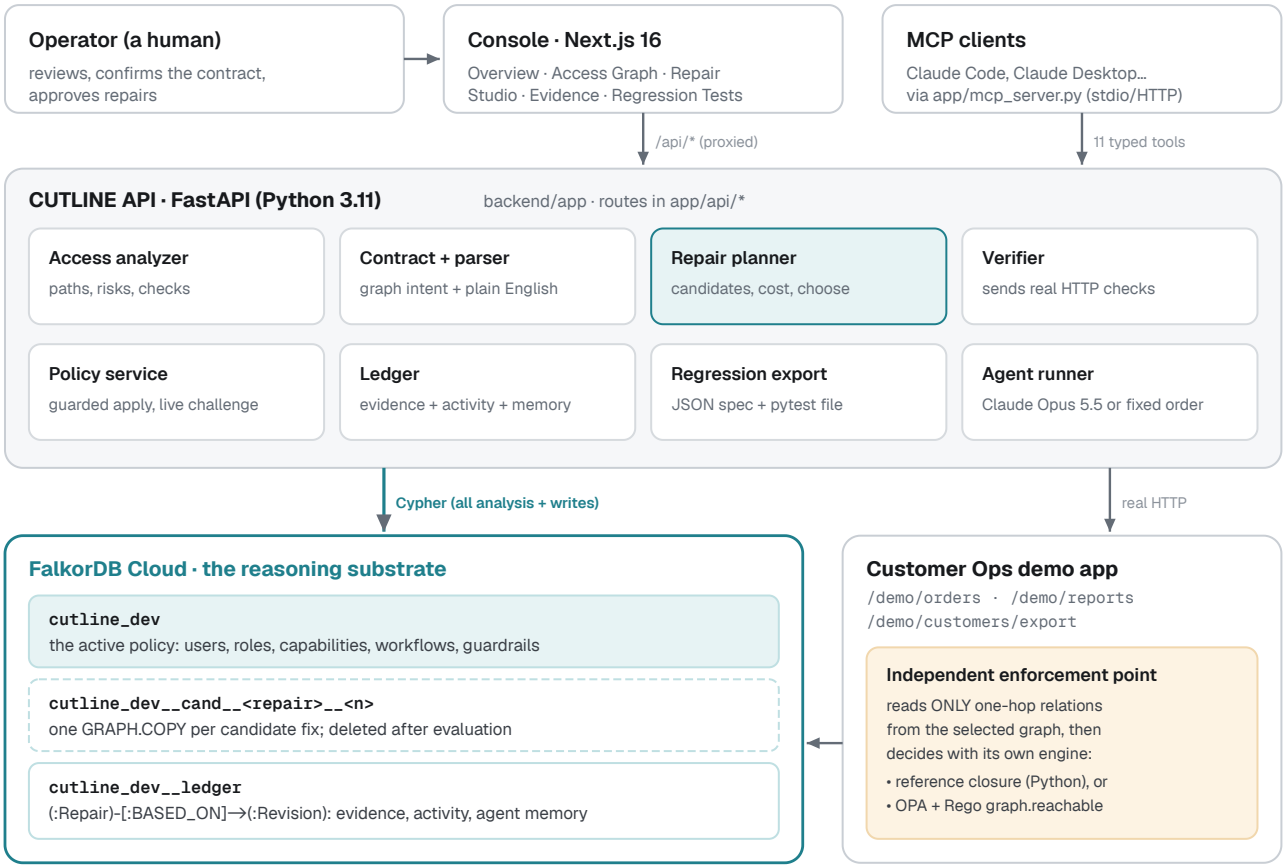


Figure 11. Architecture. Everything the product *reasons* with lives in FalkorDB. The planner and the enforcement point are deliberately independent: they read the same graph through different code, so a bug in one shows up as a disagreement instead of a false "pass".

The design principle, straight from the planning report: **the AI interprets and explains; FalkorDB holds and reasons over the permission state; a deterministic planner and verifier make the security-critical decisions; an independent engine confirms that the graph's predictions match real enforcement.** The AI is never the security proof.

5.2 The graph model

NODE LABEL	KEY PROPERTIES	MEANING
`User`	id, name, department, principal_type (human/service)	A principal
`Role`	id, name, origin (idp / cutline / challenge)	A permission bundle; *origin* records who created it
`Capability`	id, sensitivity, method, endpoint	An operation on the app, mapped to its HTTP endpoint
`Resource`	id, classification	The data a capability acts on
`Workflow`	id, name, criticality	A business process that must survive

NODE LABEL	KEY PROPERTIES	MEANING
`Guardrail`	id, description	A security rule that must hold
`Policy`	id='active', revision, updated_at, last_change	The revision boundary used for staleness

```
(User)-[:ASSIGNED_TO]->(Role)          (Workflow)-[:REQUIRES]->(Capability)
(Role)-[:INHERITS]->(Role)             (User)-[:PERFORMS]->(Workflow)
(Role)-[:GRANTS]->(Capability)         (Guardrail)-[:FORBIDS]->(Capability)
(Capability)-[:ACTS_ON]->(Resource)    (Guardrail)-[:APPLIES_TO]->(User)
```

“A INHERITS B” means A receives B’s grants. IDs are unique (a range index plus a `GRAPH.CONSTRAINT UNIQUE` on User, Role, and Capability). Business intent (workflows and guardrails) lives in the same graph as the permissions, so “which workflow does this edit break?” is just another graph query.

5.3 The Cypher that does the work

All queries live in `backend/app/db/queries.py`. Values are always passed as parameters, never pasted into the query text. That is the standard defence against injection.

Every access path from one person to one capability

```
MATCH p = (u:User {id: $principal})-[:ASSIGNED_TO]->(:Role)-[:INHERITS*0..6]->(:Role)
          -[:GRANTS]->(c:Capability {id: $capability})
RETURN DISTINCT [n IN nodes(p) | n.id] AS node_ids, [r IN relationships(p) | type(r)] AS rel_types
```

In words: start at the person, step to any role they’re assigned to, follow 0 to 6 inheritance links, and end at a role that grants the capability. Return every distinct route as a list of node IDs. A direct grant is the 0-hop case. This one query produces the witness paths shown everywhere in the UI.

The same thing for many (person, capability) pairs in one round trip

```
UNWIND $pairs AS pair
MATCH p = (u:User)-[:ASSIGNED_TO]->(:Role)-[:INHERITS*0..6]->(:Role)-[:GRANTS]->(c:Capability)
WHERE u.id = pair.principal AND c.id = pair.capability
RETURN DISTINCT u.id AS principal, c.id AS capability, [n IN nodes(p) | n.id] AS node_ids, ...
```

The full effective-access matrix

```
MATCH (u:User)-[:ASSIGNED_TO]->(:Role)-[:INHERITS*0..6]->(:Role)-[:GRANTS]->(c:Capability)
RETURN DISTINCT u.id AS principal, c.id AS capability
```

In words: who can do what, for everyone. Comparing this before and after a candidate is how CUTLINE detects “no new privilege” and “no collateral loss”.

What the independent enforcement point is allowed to read

```
MATCH (a)-[r]->(b) WHERE type(r) IN ['ASSIGNED_TO', 'INHERITS', 'GRANTS'] RETURN type(r), a.id,
b.id
```

In words: just the raw one-step connections. The enforcement engine works out reachability itself, so it does not share the planner’s path query and can catch its mistakes.

5.4 The repair planner

The planner (`backend/app/services/repair_planner.py`) is deterministic and bounded. Its budget is explicit: at most **24 candidates**, cuts of at most **3 edges**, inheritance depth of at most **6**.

Step 1: generate candidates, in three families

FAMILY	WHAT IT TRIES	EXAMPLE
`single_cut`	Remove one edge that lies on a forbidden path (an assignment, an inheritance, or a grant)	Detach Shared Ops from Support
`multi_cut`	The smallest sets of edges (size ≤ 3) that together cut *every* forbidden path. In computer-science terms these are "minimal hitting sets".	Detach Shared Ops *and* Export Helper from Support
`scoped_role_migration`	Create a new `<role>_safe` granting exactly what the subjects should keep; move them onto it; remove them from every role on a forbidden path	Support (safe), the winner

Step 2: evaluate each candidate on its own graph

For each candidate: `GRAPH.COPY` the active policy into a fresh graph, apply the candidate's edits there, then ask FalkorDB three questions. Is every must-deny pair now unreachable? Is every must-allow pair still reachable? Did anyone gain a (person, capability) pair they didn't have before? The outcome is one of:

`PASS` `FAIL_FORBIDDEN_PATH` + the path still open `FAIL_REQUIRED_WORKFLOW` + the workflow that broke `FAIL_NEW_PRIVILEGE` + the new path

If `GRAPH.COPY` is unavailable, the planner falls back to rebuilding the copy from a snapshot ("snapshot-replay"). The UI shows which method was used for each candidate.

Step 3: choose by a transparent cost

```
cost = Σ edit weights                CREATE_ROLE = 2, every other edit = 1
     + 2 × shared-role edits         "blast radius": edits that change a pre-existing role
                                     that people outside the contract can reach
     + 3 × undeclared losses         "collateral": capabilities lost that nobody declared
```

CUTLINE picks the **cheapest candidate that passes**. The claim is deliberately modest: "the lowest-cost verified candidate among the bounded set generated", **not** "the globally optimal redesign".

Step 4: be honest when there is no good answer

If nothing passes within budget, the plan status is `INFEASIBLE_WITHIN_BUDGET` . CUTLINE only removes or restructures existing access; it **never invents new grants** to satisfy a requirement. It also reports residual risk in plain words (e.g. the note about the old Support role).

5.5 Verification: two independent judges

	PLANNER (PREDICTS)	ENFORCEMENT POINT (DECIDES)
Reads	The full policy graph, with variable-length Cypher	Only one-hop relations from the selected graph

	PLANNER (PREDICTS)	ENFORCEMENT POINT (DECIDES)
Decides with	FalkorDB reachability	`reference` (a Python fixpoint closure) **or** OPA with `policies/cutline_authz.rego` using Rego's `graph.reachable`
Used for	Proposing and predicting	Actually serving `/demo/orders`, `/demo/reports`, `/demo/customers/export`

Requests carry two headers. `X-Cutline-Principal` selects who is asking (a demo selector, **not** real authentication). `X-Cutline-Policy` is either `active` or `candidate:<repair_id>`, so a candidate can be tested before it is applied. Each response reports the decision, the engine, a decision ID, and the graph that was read. **A check passes only if the HTTP decision equals the contract's expectation and also equals FalkorDB's prediction.**

5.6 Revisions, staleness, and the atomic write

Every plan records the policy **revision** (a counter) and a **content hash** (`sha256` over the canonical policy edges) it was built against. A plan is **stale** if the revision moved *or* the hash drifted. The hash catches someone editing the graph directly in FalkorDB Browser without bumping the revision. Approve and apply both refuse stale plans with `409 STALE_PROPOSAL`.

Applying is **one Cypher statement**. Every precondition is a `MATCH` placed before any write, including the revision check. If the revision moved or an expected edge is missing, the statement matches zero rows and writes nothing. Otherwise every change commits together. A single FalkorDB query is atomic.

```
-- generated for the Support (safe) migration; preconditions first, writes after
MATCH (pol:Policy {id: 'active'}) WHERE pol.revision = $expected_revision  -- compare-and-set
MATCH (n0:Capability {id: $n0_id})       -- generate_reports
MATCH (n1:Capability {id: $n1_id})       -- view_orders
MATCH (n2:User {id: $n2_id})             -- maya
MATCH (n3:User {id: $n3_id})             -- noah
MATCH (n4:Role {id: $n4_id})             -- support
MATCH (n2)-[x5:ASSIGNED_TO]->(n4)         -- edges to delete must exist
MATCH (n3)-[x6:ASSIGNED_TO]->(n4)
SET pol.revision = pol.revision + 1, pol.updated_at = $ts, pol.last_change = $change
MERGE (cr0:Role {id: $cr0_id}) ON CREATE SET cr0.name = $cr0_name, cr0.origin = 'cutline'
MERGE (cr0)-[:GRANTS]->(n0)  MERGE (cr0)-[:GRANTS]->(n1)
MERGE (n2)-[:ASSIGNED_TO]->(cr0)  MERGE (n3)-[:ASSIGNED_TO]->(cr0)
DELETE x5  DELETE x6
RETURN pol.revision AS revision
```

5.7 The life of a repair plan

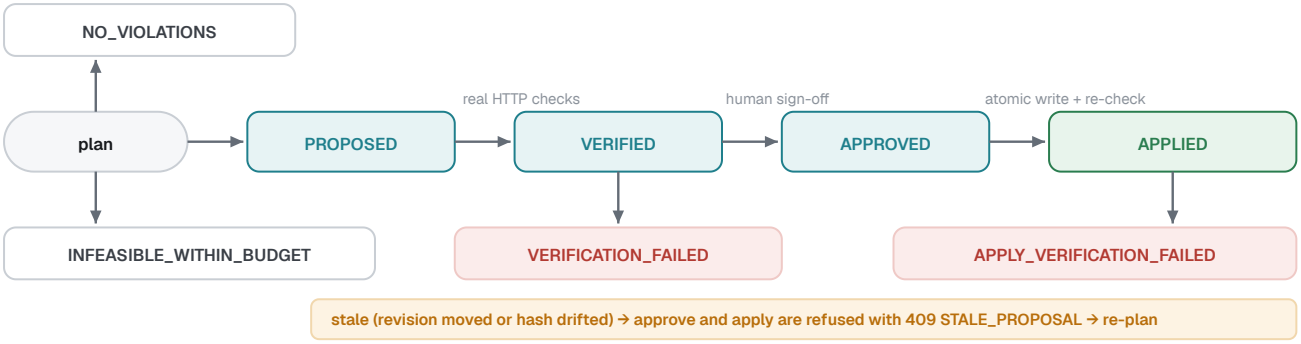


Figure 12. Repair statuses. “Stale” is not a status but a live check, re-computed on every read, that blocks the two steps that change things.

5.8 The agent and MCP (the Track 01 part)

`POST /api/agent/investigate` runs an agent over 11 typed tools (`backend/app/agent/tools.py`). With an `ANTHROPIC_API_KEY` configured, **Claude Opus 5.5** chooses which tools to call in a tool-use loop (with prompt caching and a refusal fallback). Without a key, the same tools run in a fixed order and the response is labelled `mode: "deterministic"` . The same tools are also published as an **MCP server** (`backend/app/mcp_server.py` , MCP Python SDK v2), so any MCP client can drive CUTLINE.

TOOL	KIND	WHAT IT DOES
`inspect_policy` , `find_risks` , `trace_access` , `check_access`	read	Multi-hop investigation of the policy graph
`recall_history`	read	Memory across sessions, read from the FalkorDB ledger graph
`get_contract`	read	The derived contract, or turn plain English into a proposed contract
`propose_repair` , `verify_repair`	write (isolated)	Plan on candidate graphs; run the runtime proof
`get_evidence` , `export_regression_tests`	read	Revision-bound evidence; the pytest suite
`apply_repair`	**destructive**	Only works after a human approved the plan in the console

THE SAFETY DESIGN IN ONE LINE

The model never writes Cypher, never sees graph names, and can never approve. It chooses *which* investigation to run; the deterministic planner and the runtime verifier decide whether a repair is safe.

5.9 Security design

THREAT	CONTROL
The AI invents or drops business requirements	A plain-English contract is only a proposal; the operator confirms it; workflows in the graph are always merged in
The agent changes the policy directly	Only typed, allow-listed edits; no Cypher tool; apply needs human approval

THREAT	CONTROL
A plan is applied to an outdated policy	Revision compare-and-set inside the write, plus content-hash drift detection
A migration half-applies	A single atomic statement with all preconditions matched before any write
A repair quietly grants new access	Effective-access diff on the candidate graph → `FAIL_NEW_PRIVILEGE`
The graph and the running app disagree	Verification fails; the enforcement point reads only one-hop relations through separate code
Arbitrary graph access via request headers	`X-Cutline-Policy` resolves only `active` or a known repair's candidate graph
Real customer data	Synthetic dataset only; the principal header is a demo selector, not authentication

5.10 What it supports, and what it deliberately doesn't

SUPPORTED

Allow-only RBAC with role inheritance; named principals and capabilities; bounded depth (default 6 inheritance hops).

NOT MODELLED

Deny rules, conditions/ABAC, time-bound grants, resource-scoped grants, cloud IAM policy languages. These are not silently approximated: CUTLINE only claims what its declared model can prove.

5.11 Why FalkorDB is essential (the qualifying argument)

WORK	DONE BY FALKORDB AS
Effective-access reachability and multi-path witnesses	Variable-length Cypher traversal
Business intent (workflows, guardrails) and the derived contract	Graph nodes and relationships
Candidate isolation	`GRAPH.COPY` : one throw-away graph per candidate
Counterexamples (path still open, workflow lost, new privilege)	Re-querying the candidate graph
Atomic, revision-guarded application	A single parameterised Cypher statement
Evidence ledger and cross-session agent memory	The `cutline_dev__ledger` graph
What the enforcement point reads at runtime	One-hop relations from the selected graph

Proof, not just a claim: `tests/test_falkordb_unavailable.py` points the app at a dead port and asserts that every analysis and repair endpoint returns `503 FALKORDB_UNAVAILABLE` and the protected app fails closed (denies everything). There is no fallback store.

PART 6

A tour of the codebase LEVEL 3

About 5,300 lines of Python (backend, tests, benchmarks, policy) and 9,600 lines of TypeScript (console), all under `D:\CUTLINE`.

Top level

`README.md`	The judge-facing document: problem, why FalkorDB, graph model, exact Cypher, algorithm, how to run, benchmarks, demo script, security, AI disclosure
`flow.md`	The build log: what was done, in what order, what remains (kept current so work can stop and resume)
`docker-compose.yml`	Full local stack: FalkorDB + OPA 1.21.2 + API + console
`docs/API_CONTRACT.md`	The source-of-truth contract between backend and frontend: scenario, graph names, every REST shape
`docs/DEPLOY.md`	How to deploy for judging (FalkorDB Cloud, API host, console host, keep-alive)
`docs/benchmarks.md`	Generated benchmark report (real FalkorDB Cloud run)

Backend: `backend/`

PATH	WHAT IT DOES
`app/main.py`, `container.py`, `config.py`, `errors.py`	FastAPI app startup, wiring of services, settings from `.env`, error format
`app/api/` (graph, repairs, agent, system, deps)	The REST routes under `/api`
`app/db/queries.py`	Every Cypher query, in one place
`app/db/graph_store.py`	Snapshots and hashing, effective access, witness paths, `GRAPH.COPY` isolation, the composite atomic writer, the ledger graph
`app/db/scenario.py`, `falkordb_client.py`	The Customer Ops seed data and live-challenge mutations; the FalkorDB connection
`app/services/access_analyzer.py`	Paths, risks, access checks
`app/services/contract.py`, `contract_parser.py`	Deriving the contract from the graph; plain English → contract
`app/services/repair_planner.py`	Candidate generation, isolated evaluation, cost model, choice, residual notes
`app/services/repair_service.py`, `verifier.py`	The plan → verify → approve → apply lifecycle; runtime HTTP checks
`app/services/policy_service.py`	Revision and hash, guarded apply, live-challenge mutations, reset
`app/services/authorization.py`	The independent enforcement engines (reference closure and OPA client)
`app/services/ledger.py`, `regression_export.py`	Evidence and activity in FalkorDB; JSON and pytest export

PATH	WHAT IT DOES
`app/demo_app/customer_ops.py`	The synthetic Customer Ops API (`/demo/*`) that gets protected
`app/agent/tools.py` , `runner.py` ; `app/mcp_server.py`	The 11 typed tools; the Claude / deterministic agent loop; the MCP server
`app/models/`	Pydantic data shapes (graph, policy, repair) mirroring the API contract
`policies/cutline_authz.rego`	The OPA policy (uses `graph.reachable`)
`tests/`	Planner, paths, verification, staleness, regression export, parser, agent, MCP, OPA, FalkorDB-unavailable, source hygiene; `memory_store.py` is a test double that mirrors the Cypher semantics
`benchmarks/`	10 scenarios with known correct outcomes + runner that writes `docs/benchmarks.md`
`scripts/falkordb_smoke.py`	One-command check that a FalkorDB endpoint supports every Cypher shape CUTLINE uses

Frontend: `frontend/` (the console)

PATH	WHAT IT DOES
`src/app/` routes: `/` , `/graph` , `/repair` , `/evidence` , `/tests`	The five screens: Overview, Access Graph, Repair Studio, Evidence, Regression Tests
`src/components/graph/`	The interactive permission graph (React Flow), custom nodes and edges, toolbar
`src/components/repair/`	Contract panel, candidate list, migration edits, verification table, apply dialog, agent run log, before/after comparison
`src/components/inspector/` , `evidence/` , `tests/` , `shell/` , `ui/`	Side inspectors, evidence ledger, test list, app shell and sidebar, shared UI primitives
`src/lib/`	API client, TypeScript types (mirroring the contract), graph layout, React Query hooks
`scripts/screenshot.mjs` , `e2e-live.mjs`	Design-review screenshots (mock data); live end-to-end walk of the full demo against the real API
`qa-fixtures/` , `qa-screenshots*/`	Mock API responses for design review; screenshots from both runs

Tech stack

LAYER	CHOICE (PINNED VERSIONS)
Graph database	FalkorDB Cloud (ap-south-1), client `falkordb 1.8.0`
API	Python 3.11 · `fastapi 0.143.2` · `uvicorn 0.54.0` · `pydantic 2.14.1` · `httpx 0.28.1`
Agent / MCP	`anthropic 1.14.0` (Claude Opus 5.5) · `mcp 2.3.0` (MCP Python SDK v2)
Independent policy engine	OPA 1.21.2 with Rego, or the built-in reference engine
Console	Next.js 16.4.1 · React 19.3 · Tailwind CSS v4 · React Flow (`@xyflow/react` 12) · TanStack Query 5 · Motion · Geist font

LAYER	CHOICE (PINNED VERSIONS)
Testing	`pytest 9.1.1` · Playwright 1.64 for browser QA

Why Next.js rather than the React + Vite the planning report suggested: it was the team's explicit choice. The console proxies `/api/*` to the backend server-side, so the browser never needs cross-origin access.

PART 7

What is built right now

Status as of 16 Oct 2026, 17:35 IST. Another working session was actively editing the project while this snapshot was taken, so some items may have moved forward since.

Status board

COMPONENT	STATUS	EVIDENCE
FalkorDB Cloud connection	Done	Credentials in `backend/.env`; smoke test passes on Cloud; console shows "FalkorDB connected · 25–36 ms"
Graph schema, seed, Cypher queries	Done	`queries.py`, `graph_store.py`; running live on Cloud
Path discovery (multi-hop, multi-path)	Done	4 risky paths found and drawn in the Access Graph
Repair planner (3 families, isolated graphs)	Done	10 candidates tested, 1 passed, in the live run
Runtime verification (demo app + reference engine)	Done	12/12 checks passed, candidate and post-apply
OPA engine + Rego policy	Built	Tests exist; the live-OPA tests are skipped unless an OPA server is running (OPA 1.21.2 binary was used earlier)
Approve → atomic guarded apply → post-apply verify	Done	Revision +1 on every live apply; 12/12 post-apply checks
Staleness (revision + hash) and live challenge	Done	Stale banner, apply blocked, replan cut only the new edge (1 edit)
Evidence ledger in FalkorDB; regression export	Done	Evidence and Regression Tests screens; JSON + pytest export
Plain-English contract parsing	Done	Rule-based parser (LLM parser when a key is set); confirm step in UI
Agent: 11 typed tools + MCP server	Done	Deterministic mode tested; MCP server tested
Agent: LLM mode (Claude chooses tools)	Needs key	No `ANTHROPIC_API_KEY` in `backend/.env`, so the agent runs in deterministic mode
Next.js console, 5 screens	Done	Built; design-review screenshots taken on mock data; exercised end to end by the live walk
Live end-to-end browser walk	Done	The 17:31 run captured all 11 steps, including the agent run, against the real API and FalkorDB Cloud (earlier runs at 17:13 and 17:18 had stopped at or before the agent step)

COMPONENT	STATUS	EVIDENCE
Benchmarks	Done	10 scenarios run on real FalkorDB Cloud (below)
README, API contract, deploy guide	Done	AI-disclosure section still has a placeholder for the team
Docker: backend + console images, compose file	Untested	`frontend/Dockerfile` was added at 17:32, so compose now has all its pieces, but nobody has run `docker compose up --build` yet (no Docker on this machine)
Public Git repository	Not started	`D:\CUTLINE` is not a git repository yet
Live deployment	Not started	Guide written in `docs/DEPLOY.md`
Demo video	Not started	Script ready (Part 10)

Tests

BACKEND TEST SUITE: 67 PASSED, 5 SKIPPED (26 S)

Run at 17:22 IST on 16 Oct. One earlier run in the same minute showed a single failure while the other session was mid-edit; the clean re-run after its edits landed passed.

The 5 skipped tests are the ones that need live external services, enabled by environment flags:

- **3 × real FalkorDB end-to-end** (`tests/test_falkordb_integration.py`): run with `CUTLINE_TEST_FALKORDB=1` . These have **not yet been run** in the recorded status, even though the Cloud instance now works.
- **2 × live OPA** (`tests/test_opa_engine.py`): run with an OPA server and `CUTLINE_TEST_OPA_URL` .

The other tests use an in-memory test double (`tests/memory_store.py`) that mirrors the Cypher semantics, plus live-socket tests where the exported pytest suite runs against a real HTTP server.

Benchmarks (real FalkorDB Cloud)

Ten synthetic scenarios with known correct answers, comparing CUTLINE with two deliberately simple baselines: a “greedy” repair that revokes the forbidden capability at the role that grants it, and a checker that only looks at direct grants.

METRIC	CUTLINE	BASELINE
Correct outcome (repair, refuse, or flag, as expected)	10 / 10	n/a
Safe repairs on the 7 repairable scenarios	7 / 7	greedy: 2 / 7
Forbidden access detected	11 / 12	direct-grant only: 7 / 12
Beyond-bound path caught by the independent engine	1 scenario	n/a
Median planning time	≈ 1.46 s	n/a

SCENARIO	EXPECTED	GOT	CHOSEN FIX	EDITS
The demo: two inherited paths, Finance and DSAR depend on the same roles	PASS	✓	scoped role	7
Needed and forbidden capability bundled in one role	PASS	✓	scoped role	4
Forbidden capability three hops down a private chain	PASS	✓	single cut	1
Roles inherit each other in a cycle	PASS	✓	single cut	1
Dedicated export role shared with a legitimate user	PASS	✓	single cut	1
One subject, two forbidden capabilities	PASS	✓	scoped role	5
Two subjects keep different capabilities	PASS	✓	scoped role	8
A required capability the subject never had	INFEASIBLE	✓	refuses to invent a grant	0
Guardrail already holds	NO_VIOLATIONS	✓	nothing to do	0
An 8-hop chain beyond the planner's 6-hop bound	DISAGREEMENT	✓	flagged by independent engine	0

The last row is a feature, not a failure. The planner's graph search is bounded at 6 hops for safety and speed; the independent engine is not bounded, so it notices the deeper path and the disagreement is reported rather than hidden. That is also why detection shows 11/12, not 12/12.

The live end-to-end run

`frontend/scripts/e2e-live.mjs` drives a real browser through the entire demo against the real API and FalkorDB Cloud, with no mocked requests, and fails on any server error or browser error. The 17:31 run captured all eleven steps: overview → access graph → generate repair → verify (12/12) → approve and apply → evidence → regression tests → live challenge (stale) → graph shows the new path → agent run. Two earlier runs that afternoon (17:13 and 17:18) had stopped at or before the final agent step; the other working session fixed that within minutes. The screenshots in Part 4 come from the successful run.

PART 8

What is left, and the plan to the deadline

The core product is built. What remains is mostly **packaging for submission**: a public repo, a deployment, a video, and a few loose ends. These items are not optional. Without a public repo and a demo video there is no valid submission, however good the code is.

Must do before submitting (in order)

1. **Create the public Git repository**
   The project folder is not under git yet. Initialise it, check `git status` carefully so `backend/.env` (FalkorDB password) is **not** committed (`.gitignore` already lists `.env`), push to a public GitHub repo. Committing during the window also gives judges a visible timeline of work done 15–18 Oct.

2. **Test the Docker path once**
   `frontend/Dockerfile` was added at 17:32, so `docker compose up --build` (the "complete local setup" path judges may use) should now build the whole stack. Nobody has run it yet, because this machine has no Docker. Run it once on any machine with Docker, or in a GitHub Actions job.

3. **Deploy it live**
   Follow `docs/DEPLOY.md` : FalkorDB Cloud (already running) + API container on Railway/Render/Fly + console on Vercel with `CUTLINE_API_URL` set. The console bakes the API URL in **at build time**, so set it before building (on Vercel as an environment variable; with Docker as `--build-arg CUTLINE_API_URL=…`). Then run the two smoke-test `curl` commands from the guide.

4. **Keep FalkorDB Cloud alive**
   The Free tier is **stopped after 1 day without use** (deleted after 7) and has no persistence. Point an uptime monitor at `GET /api/health` every few hours until judging ends. CUTLINE re-seeds its scenario if the graph is missing, but a *stopped* instance takes the demo down.

5. **Run the real-FalkorDB test suite and record it**
   Set `CUTLINE_TEST_FALKORDB=1` and run `pytest -m falkordb -v` (exact PowerShell commands in Part 9, step 3). Optionally start OPA and run the 2 OPA tests too. Put the results in the README.

6. **Re-run the live end-to-end walk after the last code change**
   Run `node scripts/e2e-live.mjs` (API on :8000, console on :3000) once more right before recording, so every screen in the video has just been exercised against the real stack.

7. **Record the demo video (~3 minutes)**
   Use the script in Part 10. Record from the deployed URL if possible. Show the actual HTTP 200/403 results, not just green ticks.

8. **Fill in the AI-tools disclosure and team notes**
   The README's disclosure section has a placeholder ("Team: add your own contribution notes here"). The rules require disclosure *and* that you can explain the model, queries, and architecture. Parts 4–5 of this document are your study guide.

9 **Submit before 23:59 IST on 18 Oct**
Repo URL, live URL, video, and the README (which already covers the graph model and Cypher).

Strongly recommended

1 **Turn on the LLM agent**
Add an `ANTHROPIC_API_KEY` to `backend/.env` (and the deployment secrets). Without it, the agent runs in deterministic mode, which works but undersells Track 01's "tool selection" criterion. Also consider showing CUTLINE as an MCP server inside Claude Code for 10 seconds of the video (`claude mcp add cutline -- …`).

2 **Update `flow.md`**
It is behind reality: it says publishable benchmarks still need real FalkorDB (they've been run on Cloud) and reports an older test count.

3 **Small polish**
`docs/benchmarks.md` prints the median as "1460.8000000000002 ms" (round it). `frontend/README.md` is still the default create-next-app text (replace it or delete it).

Fixed by the other session while this document was being written (17:20–17:32)

ISSUE	FIX
Edit reasons showed "â€"" instead of "—" (UTF-8 text re-saved through a Windows cp1252 tool)	Sources cleaned; new `tests/test_source_hygiene.py` fails the build if mojibake or a byte-order mark ever reappears
Re-planning after a migration created a role called `support_safe_safe`	Planner now derives the name from the role's stem: a second round gives `support_safe_2`
The live browser walk stopped before the agent-run step	The 17:31 run completed all 11 steps
`frontend/Dockerfile` was missing, so `docker compose` would fail at the console	Added at 17:32 (multi-stage Node 24 build; API URL passed as a build arg)

Stretch goals, only if everything above is done

In the planning report's priority order: (1) the judge challenge mode is already built, so polish it; (2) more benchmark scenarios, and timing on larger synthetic graphs (100 / 500 / 1,000 principals) to show the algorithm scales sensibly; (3) a GitHub-style "policy change review" artifact attached to a repair; (4) OPA decision logs shown as part of the evidence. **Avoid:** new adapters (AWS IAM, OpenFGA, Kubernetes), multi-cloud support, multi-agent architecture, or "company brain" features. The report calls these scope traps.

Day-by-day plan

Thu 16 Oct (rest of today)

- Git repo + first push
- Test the Docker build somewhere
- Real-FalkorDB tests
- Re-run live e2e walk
- Update flow.md

Fri 17 Oct

- Deploy API + console; smoke-test
- Uptime monitor on /api/health
- Anthropic key → LLM mode
- Rehearse the demo 3+ times
- Optional stretch work

Sat 18 Oct

- **Feature freeze at 12:00 IST**
- Record + edit the video
- README: disclosure, results, links
- Final check of the live URL
- **Submit well before 23:59 IST**

Risks to watch

RISK	MITIGATION
FalkorDB Cloud free instance stops or loses data	Uptime pings; automatic re-seed; the “Reset scenario” button; keep the Docker path working as a fallback for judges
The demo breaks live	Deterministic seed; one-click reset; rehearse; record the video from a known-good state
Running out of time on packaging	Packaging items first (repo, deploy, video), polish last, and respect the freeze
A judge asks about something unsupported (deny rules, ABAC, AWS)	Say so plainly and point to “Supported semantics”: CUTLINE claims only what it can prove
“Is FalkorDB really central?”	Part 5.11 table + the FalkorDB-unavailable test + the candidate graph names visible in the UI
Being unable to explain the AI-assisted code	Study Parts 4–5; walk through `queries.py` and `repair_planner.py` once yourself

PART 9

Running it yourself

On this Windows machine there is no Docker and no WSL, so FalkorDB runs in the cloud and everything else runs directly. The commands below are for **PowerShell** (VS Code's default terminal here). Use `py -3.11` or the project venv, not bare `python` (on this machine that is the Microsoft Store stub), and `curl.exe`, not `curl` (in Windows PowerShell, `curl` is an alias for a different command).

1. Backend (API)

```
# one-time setup
cd D:\CUTLINE\backend
py -3.11 -m venv .venv
.venv\Scripts\pip install -r requirements.txt
# backend\.env already holds FALKORDB_HOST / PORT / USERNAME / PASSWORD

# check the FalkorDB endpoint supports every query shape (UTF-8 avoids a Windows console error)
$env:PYTHONIOENCODING = "utf-8"
.venv\Scripts\python -m scripts.falkordb_smoke

# start the API (seeds the demo scenario on first start) → http://127.0.0.1:8000/docs
.venv\Scripts\python -m uvicorn app.main:app --port 8000
```

2. Console (second terminal)

```
cd D:\CUTLINE\frontend
npm install
npm run dev            # → http://localhost:3000 (proxies /api to http://127.0.0.1:8000)
```

3. Tests and benchmarks

```
cd D:\CUTLINE\backend
.venv\Scripts\python -m pytest                       # unit + live-socket tests

$env:CUTLINE_TEST_FALKORDB = "1"
.venv\Scripts\python -m pytest -m falkordb -v         # real FalkorDB end-to-end

.venv\Scripts\python -m benchmarks.run               # writes docs\benchmarks.md

# optional: OPA as the enforcement engine (needs the opa binary on PATH)
# in its own terminal:
opa run --server --addr 127.0.0.1:8181 policies/cutline_authz.rego
$env:CUTLINE_TEST_OPA_URL = "http://127.0.0.1:8181"
.venv\Scripts\python -m pytest tests/test_opa_engine.py
```

4. The MCP server

```
cd D:\CUTLINE\backend
.venv\Scripts\python -m app.mcp_server                                    # stdio
claude mcp add cutline -- D:\CUTLINE\backend\.venv\Scripts\python -m app.mcp_server
```

5. Quick manual check

```
curl.exe -s http://127.0.0.1:8000/api/health

# as Maya: 200 before the repair, 403 after it
curl.exe -s -X POST http://127.0.0.1:8000/demo/customers/export -H "X-Cutline-Principal: maya"

# reset the scenario back to the start
curl.exe -s -X POST http://127.0.0.1:8000/api/scenarios/customer-ops/reset
```

The REST API at a glance

System	`GET /api/health` · `GET /api/scenarios` · `POST /api/scenarios/customer-ops/reset` · `GET /api/activity`
Graph & access	`GET /api/graph` · `GET /api/policy/revision` · `POST /api/access/paths` · `POST /api/access/check` · `GET /api/risks`
Contract	`GET /api/contract` · `POST /api/contract/parse`
Repairs	`POST /api/repairs/plan` · `GET /api/repairs` · `GET /api/repairs/{id}` · `POST /api/repairs/{id}/verify | approve | apply`
Evidence & tests	`GET /api/evidence/{id}` · `GET /api/regression-tests/{id}` · `GET /api/regression-tests/{id}/export?format=json|pytest`
Demo & agent	`POST /api/demo/mutate-policy` (contractor_tools / direct_export_helper) · `POST /api/agent/investigate`
Protected demo app	`GET /demo/orders` · `GET /demo/reports` · `POST /demo/customers/export`

Errors always look like `{ "error": { "code", "message", "details" } }`. Codes include `FALKORDB_UNAVAILABLE` (503), `STALE_PROPOSAL` / `NOT_VERIFIED` / `NOT_APPROVED` (409), and `DEMO_MODE_DISABLED` (403). Full shapes: `docs/API_CONTRACT.md`, and interactive docs at `/docs`.

PART 10

Presenting it to judges

Lead with the failure, not the architecture. “An AI-powered graph-based security agent using MCP, OPA, FalkorDB and FastAPI” sounds like a parts list. “It removes dangerous permissions without breaking the work people still need to do, and proves it” sounds like a product.

The three-minute demo script

TIME	SHOW	SAY
0:00–0:20	`POST /demo/customers/export` as Maya → **200**, 1,000 records	“Maya is a support agent. She should look up orders and run reports, never bulk-export customers. Right now, she can.”
0:20–0:40	Access Graph → select `export_customers`; two amber routes	“She gets it through two independent inherited paths. And Finance and the DSAR bot legitimately use the very same roles.”
0:40–1:00	Repair Studio → contract (optionally type it in English → Confirm)	“Two things must be denied, six must keep working. The graph already knows the workflows.”
1:00–1:25	**Generate safe repair**; open the rejected candidates	“The obvious fixes fail, and CUTLINE proves why on an isolated copy: Finance close breaks, the other path stays open, DSAR fulfilment breaks.”
1:25–1:50	The Support (safe) migration, Before/After toggle	“A new narrow role with exactly what support needs. Seven edits. All eight constraints hold, and no new access for anyone.”
1:50–2:15	**Run verification**: the 12 real HTTP results	“Not a green tick: real requests. Maya export 403. Maya orders 200. Felix export 200. DSAR bot export 200.”
2:15–2:25	**Apply** with approval; revision +1	“A human approves. One atomic write, guarded by the revision. Checks re-run on the live policy.”
2:25–2:45	**Live challenge** → stale banner → Re-plan	“Now someone sneaks in a new path. The proof goes stale instantly, apply is blocked, and CUTLINE replans with a one-edit fix.”
2:45–3:00	Regression Tests → Export pytest; health shows FalkorDB	“Every repair ships with executable tests. FalkorDB isn’t our visualisation layer, it’s the state space CUTLINE reasons over. Remove it and nothing works.”

Questions judges are likely to ask, with answers

“What exactly does FalkorDB do that a normal database couldn’t?”

It computes reachability over variable-length inheritance chains in one query, returns every witness path, hosts a full isolated copy of the policy per candidate fix (`GRAPH.COPY`), stores the business intent and the evidence ledger as graphs, and applies the repair as one atomic, revision-guarded statement. The app returns 503 and fails closed without it; there’s a test for that.

“Isn’t the AI just guessing a fix?”

No. The AI only chooses which typed tool to call. Candidate generation, evaluation, and the safety decision are deterministic code plus FalkorDB queries, and the final word comes from real HTTP requests decided by an independent engine. The AI can’t write Cypher and can’t approve.

“Is the repair optimal?”

We claim the lowest-cost *verified* candidate among a bounded, explicitly budgeted set (24 candidates, cuts up to 3 edges, depth 6), under a published cost model. Not a global optimum, and we say so.

“Why do you need OPA if FalkorDB already knows the answer?”

To check ourselves. The planner and the enforcement point read the same graph through different code. If they ever disagree, verification fails instead of reporting a false pass. OPA is the verifier, never the planner.

“What if nothing can satisfy the requirements?”

The plan comes back `INFEASIBLE_WITHIN_BUDGET`. CUTLINE never invents new grants to make constraints pass. The benchmark includes exactly this case.

“What happens beyond 6 hops?”

The planner’s search is bounded for safety and speed, but the independent engine isn’t, so it catches deeper paths and the disagreement is reported. That’s benchmark scenario 10.

“What if someone changes the policy between approval and apply?”

The write itself compares the revision. If it moved, zero rows match and nothing is written. A content hash also catches edits made directly in FalkorDB Browser that didn’t bump the revision.

“How is this different from AWS IAM Access Analyzer or SpiceDB?”

They model, check, and explain access. CUTLINE is a remediation planner: given what must be denied and what must survive, it redesigns the structure, rejects unsafe fixes with counterexamples, proves the result against a running app, and expires its own evidence when the policy changes.

“Does it support real IAM systems?”

Not yet, deliberately. The MVP supports allow-only RBAC with inheritance. Deny rules, conditions, and cloud policy languages are listed as unsupported rather than approximated. Adapters would be the post-hackathon path, ending in a CI gate that reviews every policy pull request.

“How much of this did AI write?”

Answer honestly and match the README disclosure: the build used Claude Code substantially, and the in-app agent calls the Claude API. Then show you understand it by walking through the path query and the atomic apply query from Part 5.

THE CLOSING LINE

“One dangerous capability, two hidden graph paths, the obvious fixes rejected with proof, one structural migration that keeps every workflow alive, twelve real requests proving it, and a live policy change it catches and repairs. That’s CUTLINE.”